L-Int: Konkrete und abstrakte Syntax

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Erinnerung Lispy in der Sprachstufe L-Int

```
1                       ;; 1 (Integer)
(+ 1 2)                 ;; 1 + 2
(+ 1 2 3 4)             ;; 1 + 2 + 3 + 4
(- 10 (+ 3 5))          ;; 10 - (3 + 5)
(- (+ 1 2) (- 3 4))     ;; (1 + 2) - (3 + 4)
```

Konkrete Grammatik für L-Int

```
eform ::= '(' '+' eform eform+ ')'
        | '(' '-' eform eform+ ')'
        | INT
INT   ::= [0-9]+

lint  ::= eform EOF
```

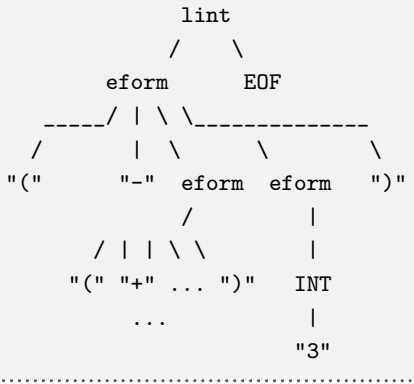
Hinweis: Pipeline, implizite Token, Whitespace

Beispiel konkrete Syntax

```
eform ::= '(' '+' eform eform+ ')' | '(' '-' eform eform+ ')' | INT
INT   ::= [0-9]+
lint  ::= eform EOF
```

Text: `(- (+ 1 2) 3)`

Parse-Tree:



Konkrete Grammatik - Eigenschaften

- Die konkrete Grammatik legt fest:
  - Welche Eingabe-Strings **syntaktisch** gültige Programme sind
  - Struktur des **Parse-Trees**

- Aber:
  - Parse-Tree enthält **"technische" Details**:
    - Klammern, `EOF`, ...
    - Hilfs-Nonterminale (`eform`)
  - Nicht optimal als interne Repräsentation für die Weiterverarbeitung

Konkrete vs. abstrakte Grammatik

- **Konkrete Grammatik**:
  - Wie der Code “auf dem Papier” aussieht
  - Enthält Syntaxdetails (Klammern, Keywords, Vorrangregeln)

- **Abstrakte Grammatik**:
  - Wie wir den Code **im Compiler** modellieren
  - Fokus auf Bedeutungsträger (z.B. `AddExpr`, `SubExpr`, `IntLiteral`)

Abstrakte Grammatik für L-Int

```
Expr = IntLiteral(Int)
     | AddExpr(Expr, Expr)
     | SubExpr(Expr, Expr)

Lint = Expr
```

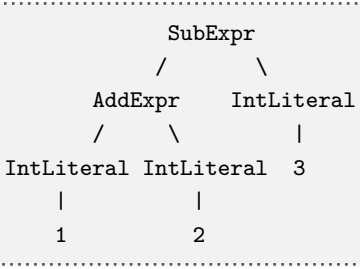
- Keine `eform`-Regel mehr
- Klammern, `EOF`, ... werden **nicht** als Knoten repräsentiert

Beispiel abstrakte Syntax (AST)

```
Expr = IntLiteral(Int) | AddExpr(Expr, Expr) | SubExpr(Expr, Expr)
Lint = Expr
```

Text: `(- (+ 1 2) 3)`

AST:



AST in Java: Definition

```
Expr = IntLiteral(Int) | AddExpr(Expr, Expr) | SubExpr(Expr, Expr)
Lint = Expr
```

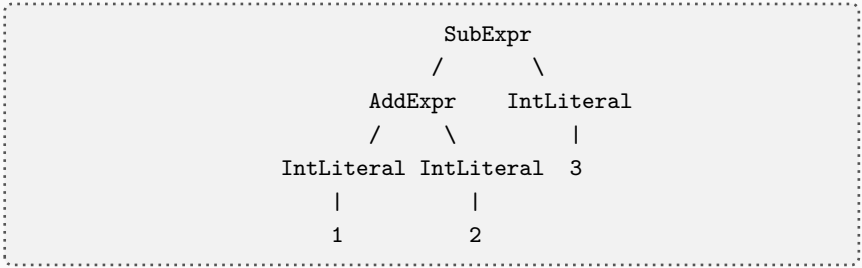
```java
public interface Expr {}

public record AddExpr(Expr left, Expr right) implements Expr {}
public record SubExpr(Expr left, Expr right) implements Expr {}
public record IntLiteral(int value) implements Expr {}
```

AST in Java: Beispielkonstruktion

Text: `(- (+ 1 2) 3)`

AST:




```
Expr e = new SubExpr(
    new AddExpr(
        new IntLiteral(1),
        new IntLiteral(2)
    ),
    new IntLiteral(3)
);
```

Wrap-up

- Zwei Sichtweisen auf L-Int:
  - Konkrete Grammatik: Syntax der Eingabe-Strings
  - Abstrakte Grammatik: AST-Struktur
- Konkrete Grammatik:
  - Regeln und Terminale
  - Akzeptiert eine Eingabe
  - Erzeugt dabei einen Parse-Tree
  - Erlaubt das Ableiten eines Parsers
- Abstrakte Grammatik:
  - Modellierung als algebraische Datentypen
  - Umsetzung in Java: Interfaces und Klassen/Records



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Last modified: *b7ac8e6 2026-11-03 lint: add screencast (grammar)*